

MODEL DOCUMENT FOR THE VMU POWER-SPLIT MODE LOGIC (HEV POWER-SPLIT ADAPTED)

Team:

Bruna Fortes
Danilo Varini
Gustavo Silva
Hugo Cardoso
Marinel Almeida

**APRIL
2026**

Contents

1. Overview	3
2. Driver Inputs	3
3. Model Interface (Entry / Exit Signals)	3
3.1 Mode Logic Inputs and Outputs	3
3.2 Other Top-level Signals	4
4. Specifications	4
5. Adaptations from the Original Model	5
5.1 Top-level Model	5
5.2 Vehicle Sensor – Power Demand (P_dem)	6
5.3 Control Subsystem	7
5.4 Mode Logic (Stateflow Chart)	7
5.5 Engine Speed Controller	8
5.6 Generator Controller	8
5.7 Engine Running Selector	9
5.8 Battery Charge Controller	10
6. What is Modeled – Mode Logic and Control	10
7. Simulation Results – Vehicle Speed by Drive Cycle	11
7.1 System Variant – Drive Cycles 1 to 6	11
7.2 Detailed Variant – Drive Cycle 6	14
8. Requirement Linking with the Requirements Manager	15

1. Overview

This document describes the HEV_powersplit_adapted Simulink model, a customised version of the Simscape Hybrid Electric Vehicle Series-Parallel example tailored to the requirements of the VMU power-split mode-logic project. The plant remains the same power-split architecture (engine on the planetary carrier, generator on the sun, motor and final drive on the ring), but the supervisory control was rewritten so that the operating modes, transition guards, and plant-facing enable signals match the requirement baseline (StandStill, EV_mode, RegenB_mode, Start_mode, ICE_mode, Hybrid_mode).

Where a subsystem is not mentioned in this document, it has been kept identical to the original example. Only adapted versions are shown in the figures; the original is referenced only in writing when necessary to explain the change.

2. Driver Inputs

The driver is represented by two top-level signals on the model canvas:

- **Kph Demand** – reference vehicle speed in kph, produced by the selected drive cycle. The drive cycles are loaded by HEV_Model_PARAM.m as UrbanCycle1...UrbanCycle6 and indexed by Drive_Cycle_Num. The Mode Logic uses the measured vehicle speed (VehSpd_kph) returned by the plant rather than the raw demand.
- **Acc** – accelerator command. It is routed to the Engine Speed Controller through the Control subsystem and is *not* consumed by the Mode Logic. The Mode Logic instead uses the measured power demand P_dem, the battery state of charge, and the engine speed to decide the active mode.

In the requirement language the driver is therefore exposed indirectly to the supervisor: vehicle speed and requested power define propulsion or regenerative operation, while Acc only conditions the engine throttle path.

3. Model Interface (Entry / Exit Signals)

The supervisory function of the adapted model is concentrated in the Mode Logic chart inside HEV_powersplit_adapted/Control/Mode Logic. Its four input signals and three enable outputs are the contract against which the requirement baseline (SwLLR01..SwLLR23) is written, so they are listed first; the remaining model-level signals follow as auxiliary context.

3.1 Mode Logic Inputs and Outputs

Mode Logic signal	Direction	Description
speed	Input	Measured vehicle speed (kph). Used by the StandStill / EV / RegenB / Motion_mode_ICE entry and exit guards.
P_dem	Input	Requested power at the wheel-side (kW), produced by the Vehicle / Sensor Speed and Power block. Drives the regen entry, the EV-to-engine demand thresholds, and the standstill stop window.
charge	Input	Battery state of charge (–). Gates the EV window via the SOC_EV_IN / SOC_EV_OUT hysteresis and selects between ICE and Hybrid behaviour through SOC_MID / SOC_LOW.
engine_speed	Input	Engine angular speed (rpm). Provides the engine-on / engine-off hysteresis (ENG_ON = 800 rpm, ENG_OFF = 790 rpm) used by Motion_mode_ICE substate transitions.

Mot_Enable	Output	Enables the Motor Controller torque path. 1 in EV_mode, RegenB_mode, Start_mode, and Hybrid_mode; 0 otherwise.
Gen_Enable	Output	Enables the Generator Controller torque/speed path. 1 in Start_mode, ICE_mode, and Hybrid_mode; 0 otherwise.
ICE_Enable	Output	Enables the engine demand and the Engine Running selector. 1 in ICE_mode and Hybrid_mode; 0 elsewhere (so the engine can be commanded off).

The four inputs (speed, P_dem, charge, engine_speed) form the complete decision domain of the chart: every transition guard in the requirement baseline references only these four signals against the named calibration constants (SPEED_STOP, SPEED_REGEN, SPEED_EV_MAX, PDEM_REGEN, PDEM_STOP_LOW/HIGH, PDEM_HYB_IN/OUT/MID/LOW, SOC_EV_IN/OUT, SOC_MID, SOC_LOW, ENG_ON, ENG_OFF). The three outputs (Mot_Enable, Gen_Enable, ICE_Enable) form the deterministic plant-facing enable pattern; their values per mode are listed in Section 6.

3.2 Other Top-level Signals

The signals below cross subsystem boundaries but are not direct inputs or outputs of the Mode Logic chart. They are listed for completeness.

Signal	Direction	Description
Kph Demand	Driver in	Reference vehicle speed (kph) coming from the selected drive cycle (UrbanCycle1–6).
Acc	Driver in	Accelerator command. Routed to the Engine Speed Controller; not used by the Mode Logic supervisor.
VehSpd_kph	Plant in/out	Measured vehicle speed (kph) delivered by the Vehicle subsystem.
TrqReqMot / TrqReqGen / Thr	Controller out	Motor torque demand, generator torque demand, and engine throttle to the Electrical and Engine subsystems.
RefSpd, vehSpd	Logged	Logged signals consumed by HEV_SeriesParallel_plot1speeds (commanded vs. measured).

4. Specifications

The numerical specifications of the adapted model are taken from Scripts_Data/HEV_Model_PARAM.m and are unchanged with respect to the original example. The most relevant values for the project are:

Parameter	Value	Source / Note
Vehicle mass	1200 kg	HEV_Param.Vehicle.Mass = 600*2
Tire radius	0.30 m	HEV_Param.Vehicle.Tire_Radius
Frontal area	2.16 m ²	HEV_Param.Vehicle.Frontal_Area
Aerodynamic drag coefficient	0.26	HEV_Param.Vehicle.Aero_Drag_Coeff
Engine–vehicle gear ratio	1.3	HEV_Param.Vehicle.Engine_Vehicle_Gear_Ratio
Power-split ring/sun ratio	2.6	HEV_Param.Power_Split.Ratio_Ring2Sun
ICE max power	114 kW (2 × 57 kW)	HEV_Param.ICE.Max_Power
ICE speed at max power	5000 rpm	HEV_Param.ICE.Speed_At_Max_Power

ICE max speed	6000 rpm	HEV_Param.ICE.Max_Speed
Motor peak torque / power	400 N·m / $\approx$ 50.3 kW	Torque–speed LUT (peak at 1200 rpm)
Generator peak torque / power	400 N·m / $\approx$ 50.3 kW	Torque–speed LUT (peak at 1200 rpm)
Battery (system) nominal voltage	217 V	HEV_Param.Battery_Sys.Nominal_Voltage
Battery (system) rated capacity	6.9 Ah	HEV_Param.Battery_Sys.Rated_Capacity
Mode-logic sample time	0.1 s	HEV_Param.Control.Mode_Logic_TS
Engine-on / Engine-off thresholds	800 / 790 rpm	HEV_Param.Control.Engine_Start_RPM / Engine_Stop_RPM

The motor / generator peak power is reported as approximately 50.3 kW because the torque–speed lookup table holds 400 N·m up to 1200 rpm (corresponding to about 50.27 kW); above that speed the LUT enters the constant-power region.

5. Adaptations from the Original Model

The adaptation effort was concentrated in the Control subsystem and in the few plant-side blocks that had to expose new signals to it. The following subsections describe each change. Only adapted views are shown.

5.1 Top-level Model

At the top level the model was renamed from HEV_SeriesParallel to HEV_powersplit_adapted to make the project identity explicit, and a description annotation summarising the adaptations and the relevant specifications was added next to the supervisory link list. The list of links itself was rewritten so that section 3 simulates and plots vehicle speed for any of the six drive cycles, the obsolete “torque for detailed electric drive” entry was removed, and the Simscape Results Explorer entry was repaired by referencing the renamed simlog variable simlog_HEV_powersplit_adapted.

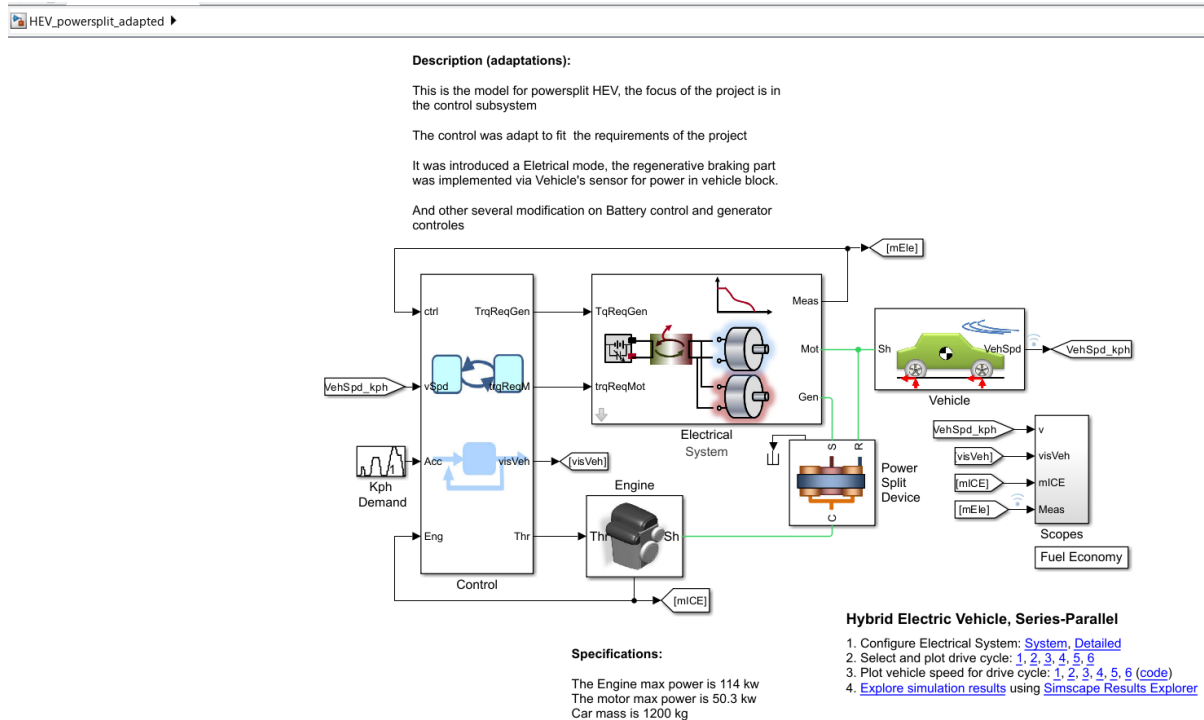


Figure 5.1: Adapted top-level diagram. The Control subsystem is the focus of the adaptations; the description annotation and the specifications block were added on the canvas.

5.2 Vehicle Sensor – Power Demand (P_dem)

In the original model the Sensor Speed and Power block computed instantaneous power but its output was terminated and never reused. The adapted model exposes that signal as the global P_dem Goto, which is then consumed by the Mode Logic and by the EV-mode transition logic. This is the mechanism that lets every transition guard be written in terms of requested power instead of a separate brake input.

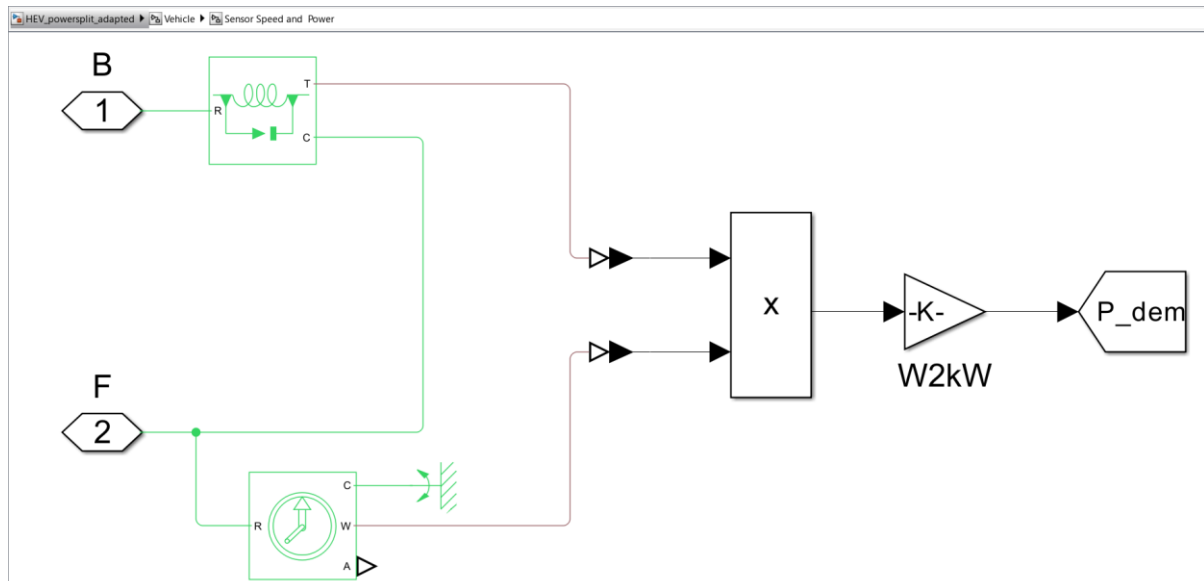


Figure 5.2: Adapted Sensor Speed and Power. The product of force and speed is converted to kW (W2kW gain) and routed to a P_dem Goto used elsewhere in the model.

5.3 Control Subsystem

The Control subsystem of the adapted model exposes a different set of inputs to the Mode Logic. The original chart used speed, brake, charge, and engine_speed; the adapted chart uses speed, P_dem, charge, and engine_speed. The brake input was removed because regenerative braking is now triggered by negative P_dem rather than by a separate brake signal. The Battery Charge Controller is now part of the live signal path: in the original model its output was terminated inside the Generator Controller, but in the adapted model the previously inactive “Alternate logic if engine can be turned off” has been wired up so that the Battery Charge Controller’s torque demand reaches the generator (see Sections 5.6 and 5.8).

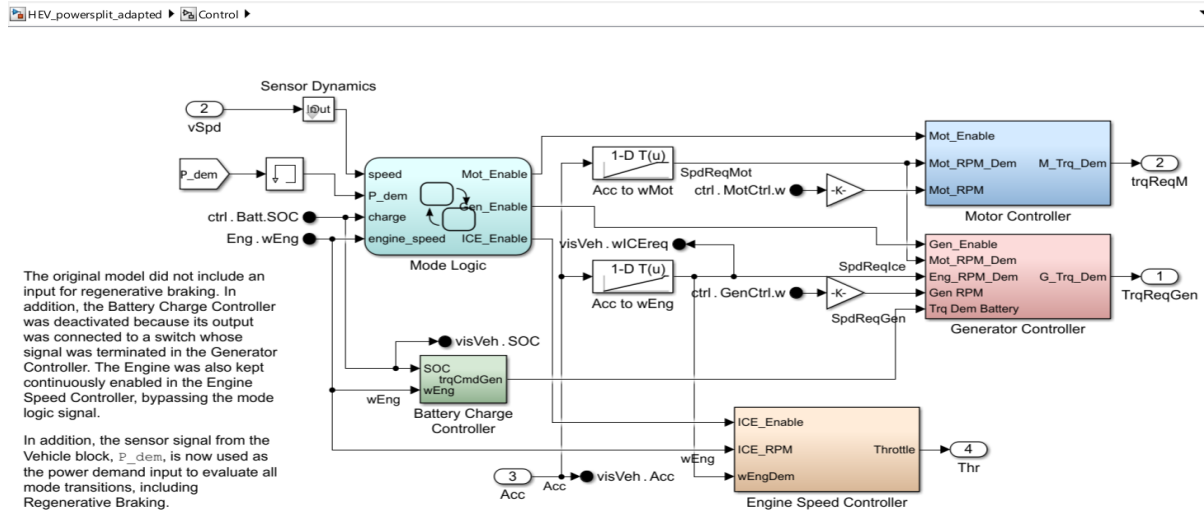


Figure 5.3: Adapted Control subsystem. The Mode Logic now receives speed, P_dem, charge, and engine_speed and outputs Mot_Enable, Gen_Enable, and ICE_Enable to the motor, generator, and engine controllers. The on-canvas annotation summarises the adaptations applied to this subsystem.

5.4 Mode Logic (Stateflow Chart)

The Stateflow chart was rewritten from scratch to match the requirement baseline. The original chart had a Brake_mode driven by a brake signal and a Motion_mode containing Start_mode, Normal_mode, Cruise_mode, and Accelerate_mode, with internal speed buffering. The adapted chart has the four top-level states StandStill, EV_mode, RegenB_mode, and Motion_mode_ICE, with the internal substates Start_mode, ICE_mode, and Hybrid_mode inside Motion_mode_ICE.

The new chart introduces an explicit standstill state that asserts {0,0,0} on the enables, a dedicated EV_mode that gates pure-electric operation by an SOC window with hysteresis (SOC_EV_IN = 0.37, SOC_EV_OUT = 0.35), and a regenerative-braking state entered when speed > 5 km/h and P_dem ≤ -5 kW. The engine-on / engine-off thresholds (800 / 790 rpm) and the power-demand thresholds (PDEM_HYB_IN, PDEM_HYB_OUT, PDEM_HYB_MID, PDEM_HYB_LOW) are read directly from HEV_Param.Control. The chart runs at the discrete sample time HEV_Param.Control.Mode_Logic_TS = 0.1 s.

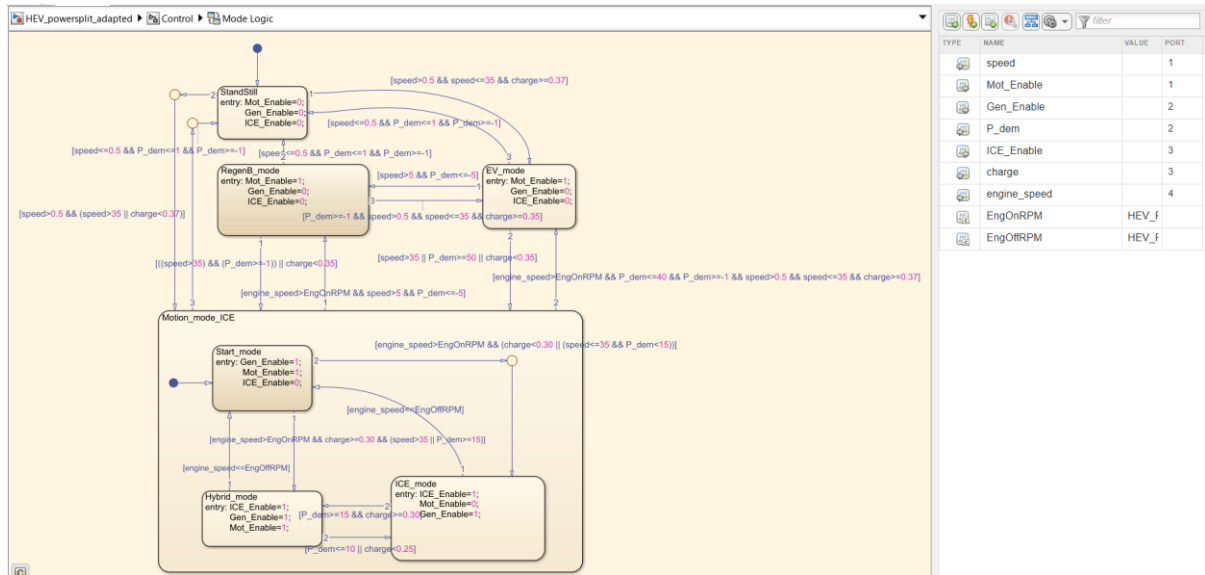


Figure 5.4: Adapted Mode Logic chart. Top-level states StandStill, EV_mode, RegenB_mode and Motion_mode_ICE replace the original Brake_mode / Motion_mode structure; Motion_mode_ICE contains Start_mode, ICE_mode and Hybrid_mode.

5.5 Engine Speed Controller

The Engine Speed Controller still uses a PI regulator with a 20 Hz low-pass filter on the measured ICE_RPM, but the Engine Idle constant fed to the Select Logic switch was changed from 800 rpm to 0 rpm. With the original value the engine speed demand was clamped to the idle speed whenever the controller was disengaged, which prevented the engine from ever being turned off. With the new value of 0, the demand collapses to zero and the throttle goes to zero, allowing the engine to actually stop. This is the mechanical condition that closes the loop with the new ICE_mode / Start_mode logic.

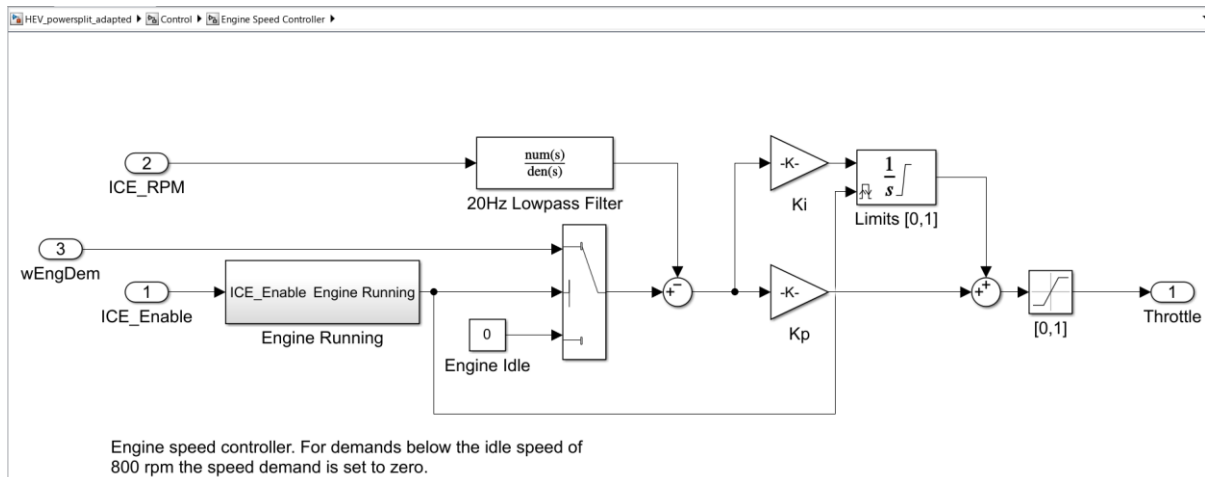


Figure 5.5: Adapted Engine Speed Controller. The Engine Idle constant changed from 800 to 0 to allow the engine to be commanded off when ICE_Enable goes low.

5.6 Generator Controller

The right-hand block labelled “Alternate logic if engine can be turned off” was already present in the original example but its output was disconnected: the model relied on the upper torque-demand path only and the alternate path was effectively dead. In the adapted version that output is wired into the Torque or Speed Control switch that feeds the Enable Torque Demand block. The switch uses the engine_running flag as its control input: when engine_running is true, it passes Trq Dem Battery, produced by the Battery Charge Controller (see Section 5.8); when

engine_running is false, it passes Tref from the Speed controller. In other words, the comparison performed by this selector is engine_running $\geq$ 0.5, not a comparison between the battery torque demand and the speed-controller torque demand. Re-enabling this path is what brings the Battery Charge Controller back into operation.

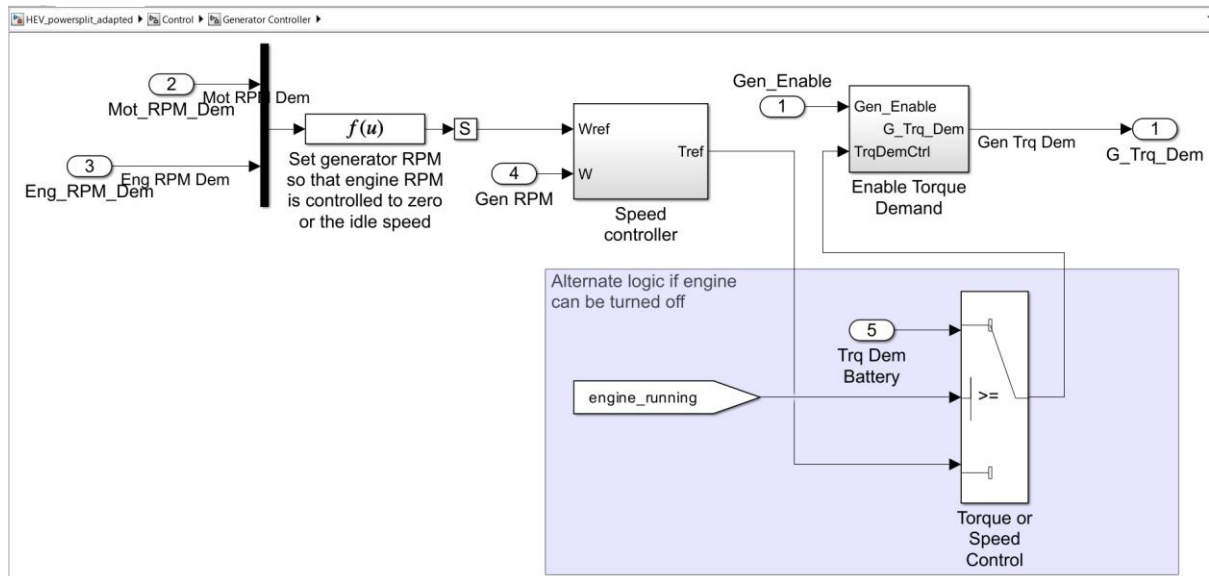


Figure 5.6: Adapted Generator Controller. The alternate torque/speed logic is now connected to the Enable Torque Demand input, gated by the engine_running flag, with Trq Dem Battery coming from the Battery Charge Controller.

5.7 Engine Running Selector

The Engine Running selector inside the Engine Speed Controller is the small subsystem that publishes the engine_running flag used by the Generator Controller and the alternate logic. In the original model the manual-enable switch was set to the upper position, which forced engine_running to a constant 1 – the engine was always considered running for control purposes. In the adapted model the switch is moved to the lower position, so engine_running tracks ICE_Enable. This is what makes the engine-off behaviour observable in the rest of the controllers.

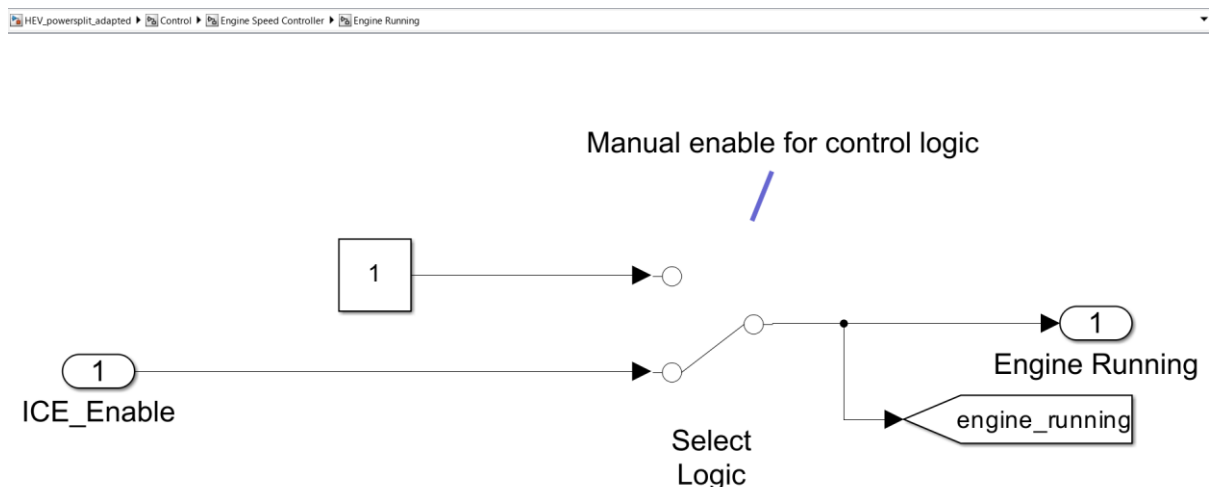


Figure 5.7: Adapted Engine Running selector. The manual-enable switch now passes ICE_Enable through, so engine_running follows the supervisory enable instead of being held at 1.

5.8 Battery Charge Controller

The Battery Charge Controller computes a generator torque demand whose role is to charge the battery from the engine when the engine is running. Internally it takes the engine speed w_{Eng} , looks up the corresponding engine torque on the ICE_Characteristics curve, scales it by the engine-to-generator gear ratio ($1/2$) and by a charging share of -0.2 (20 % of engine torque, with negative sign because the generator must absorb torque from the carrier to produce electrical power); the result is gated by a switch on SOC so that no charging request is issued when the battery state of charge is at or above 80 %.

In the original model this output ($trqCmdGen$) was routed to the Generator Controller but terminated there, because the alternate logic in the generator was disconnected. In the adapted model the alternate logic was reconnected (Section 5.6), so the Battery Charge Controller is now an active part of the control. When $engine_running$ is true, the Generator Controller selects the Battery Charge Controller output; if SOC is below 80 %, that output is the charging torque demand, and if SOC is at or above 80 %, that output is zero. When $engine_running$ is false, the Generator Controller selects T_{ref} from the Speed controller instead.

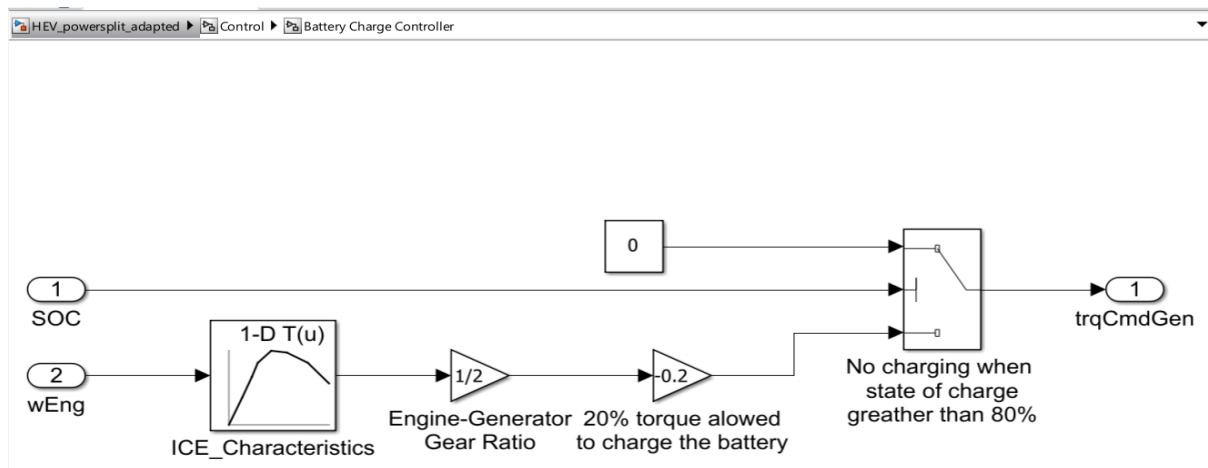


Figure 5.8: Battery Charge Controller. Engine speed w_{Eng} is mapped to engine torque, scaled by $1/2$ (gear ratio) and by -0.2 (20 % charging share), then gated so charging torque is passed only when SOC is below 80 % before being published as $trqCmdGen$.

6. What is Modeled – Mode Logic and Control

The plant captures the power-split mechanical coupling through a planetary gear set (Power Split Device), the engine and its throttle dynamics, the generator and motor (with two electrical fidelity variants – System and Detailed – selectable from the model annotation), the battery, the final drive, the wheels, and the longitudinal vehicle body. The Driver block converts the drive-cycle reference into a torque demand using a PI regulator on vehicle speed.

The adapted Mode Logic chart selects exactly one of the six modes at each 0.1 s cycle and asserts the deterministic enable pattern shown below. These patterns are consumed by the motor, generator, and engine controllers, and are also the foundation of the C implementation in `mode_logic_team.c / mode_logic_team.h`.

Mode	Mot_Enable	Gen_Enable	ICE_Enable	Role
StandStill	0	0	0	Vehicle stationary; all paths disabled.
EV_mode	1	0	0	Pure electric propulsion within the EV window.

RegenB_mode	1	0	0	Regenerative braking; motor recovers kinetic energy.
Start_mode	1	1	0	Engine being cranked / motor still propulsive.
ICE_mode	0	1	1	Engine-dominant operation.
Hybrid_mode	1	1	1	Engine plus motor assist.

The transition guards involve hysteresis on the SOC window (SOC_EV_IN / SOC_EV_OUT) and on the engine speed (ENG_ON = 800 rpm / ENG_OFF = 790 rpm), preventing oscillation around boundaries. The full list of 17 chart-level transitions and 23 software-level transitions is documented in the requirements baseline; only the supervisory inputs and outputs are exposed at this level.

7. Simulation Results – Vehicle Speed by Drive Cycle

This section collects the speed-tracking results produced by the new Section 3 of the model annotation. Each plot is generated by HEV_SeriesParallel_plot1speeds.m after a sim() call on the adapted model, and shows the commanded reference (solid blue) against the measured vehicle speed (dashed orange). The annotation in the upper-right corner of each figure reports the number of solver steps and the wall-clock duration of the simulation, useful as a rough comparison of solver effort across cycles.

All six drive cycles were simulated with the electrical system in the System variant. Drive cycle 6, being the shortest of the set, is also shown in the Detailed variant (fixed-step ode1 with the local solver enabled) to give a localised view of how much the higher-fidelity electrical model changes the speed-tracking behaviour.

7.1 System Variant – Drive Cycles 1 to 6



Figure 7.1: Drive Cycle 1, electrical system in System variant. Three trapezoidal speed plateaus up to ≈50 kph; tracking is essentially exact at the resolution of the figure.

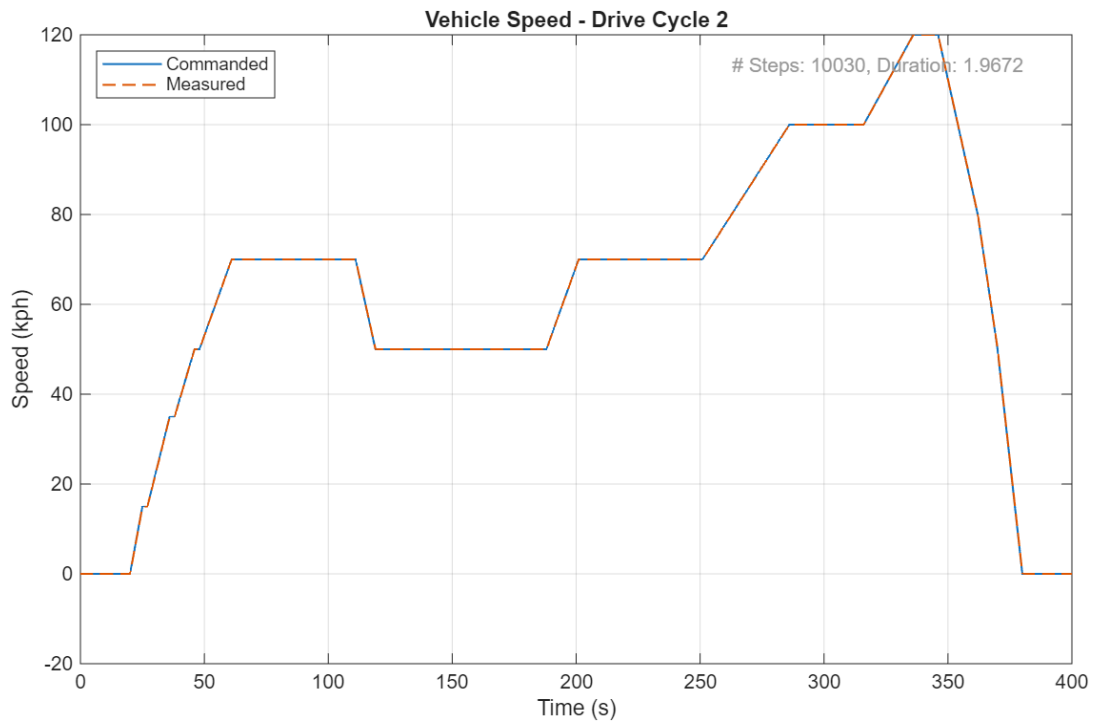


Figure 7.2: Drive Cycle 2, System variant. Slightly more aggressive transients than cycle 1; commanded and measured curves remain visually superimposed.

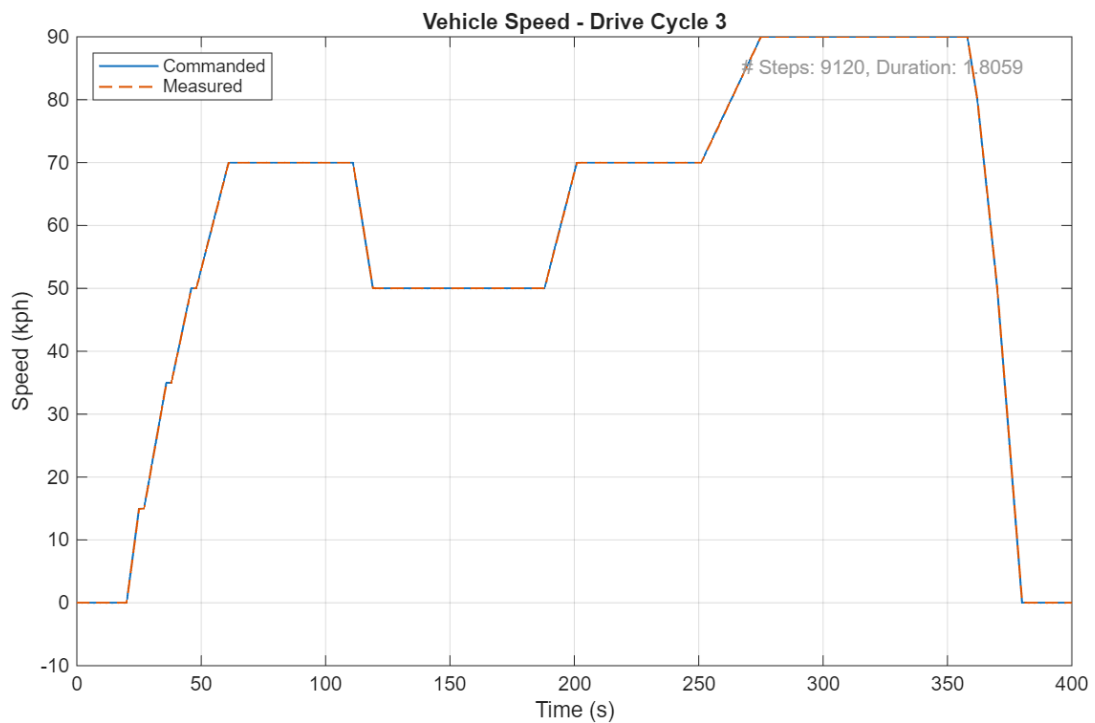


Figure 7.3: Drive Cycle 3, System variant. The shape stresses the EV / engine-supported window boundaries; speed tracking remains tight along the cycle.

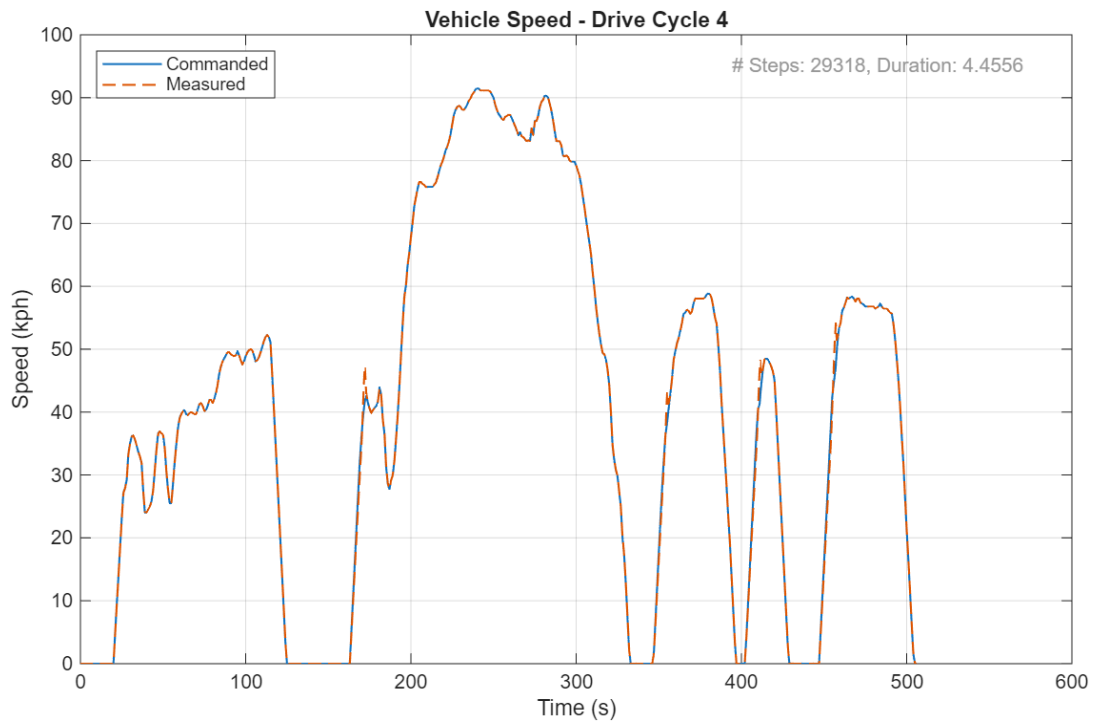


Figure 7.4: Drive Cycle 4, System variant. Higher-speed plateau exercising the upper part of the EV window and entries into Motion_mode_ICE.

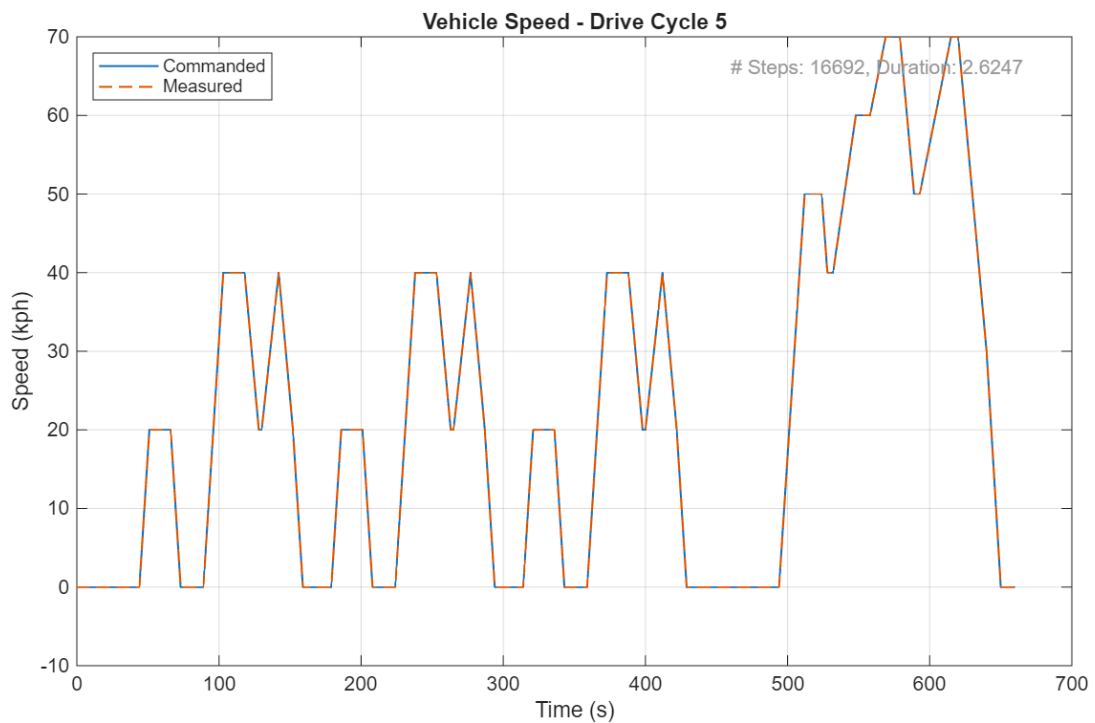


Figure 7.5: Drive Cycle 5, System variant. Mixed urban-style profile; the supervisory transitions follow without visible tracking degradation.

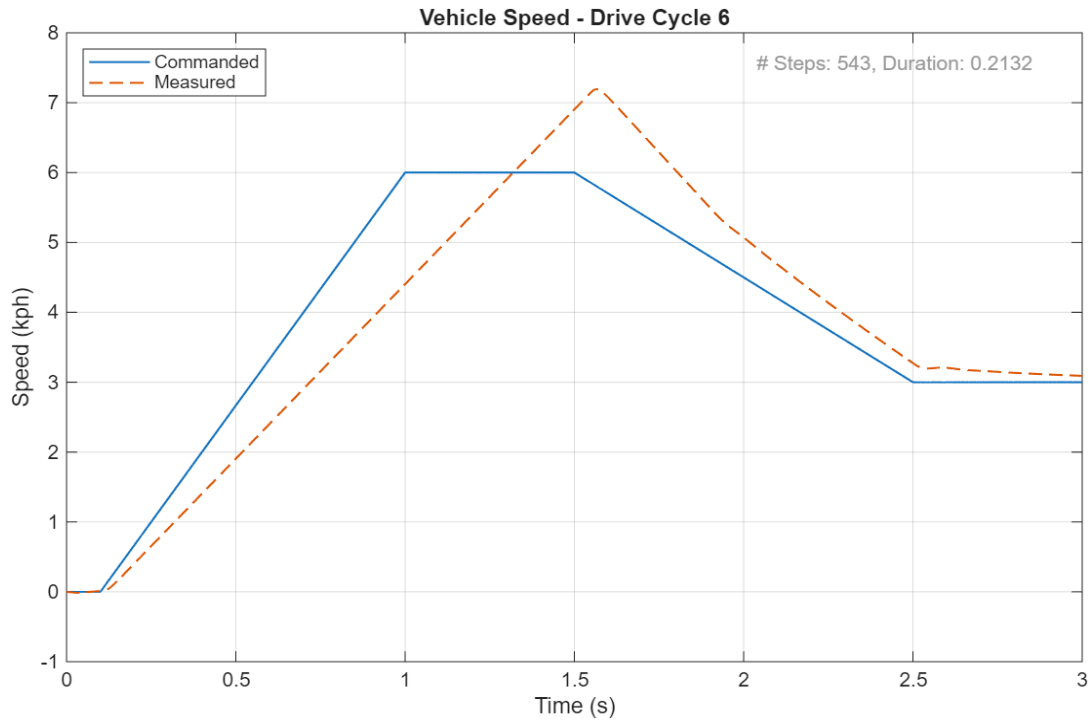


Figure 7.6: Drive Cycle 6, System variant. Very short low-speed cycle (peak ≈ 6 kph). With the variable-step solver and the System electrical variant the tracking shows visible artefacts, including a measured curve that briefly leads the commanded one – consistent with the coarse averaged dynamics of the System variant on this very short manoeuvre.

7.2 Detailed Variant – Drive Cycle 6

Drive cycle 6 is the shortest profile in the set and is therefore well suited to be re-simulated with the Detailed electrical variant. Switching to Detailed enables three-phase machine models (with explicit Meas_I current sensing in the simlog), the local Simscape solver, and a fixed-step ode1 integration. As a consequence the simulation is far more expensive ($\approx 3 \cdot 10^5$ steps and ≈ 50 s of wall-clock time on the same machine, versus $\approx 5 \cdot 10^2$ steps and ≈ 0.2 s in the System variant) but it captures the electromagnetic transients of the motor and generator that the System variant averages out.

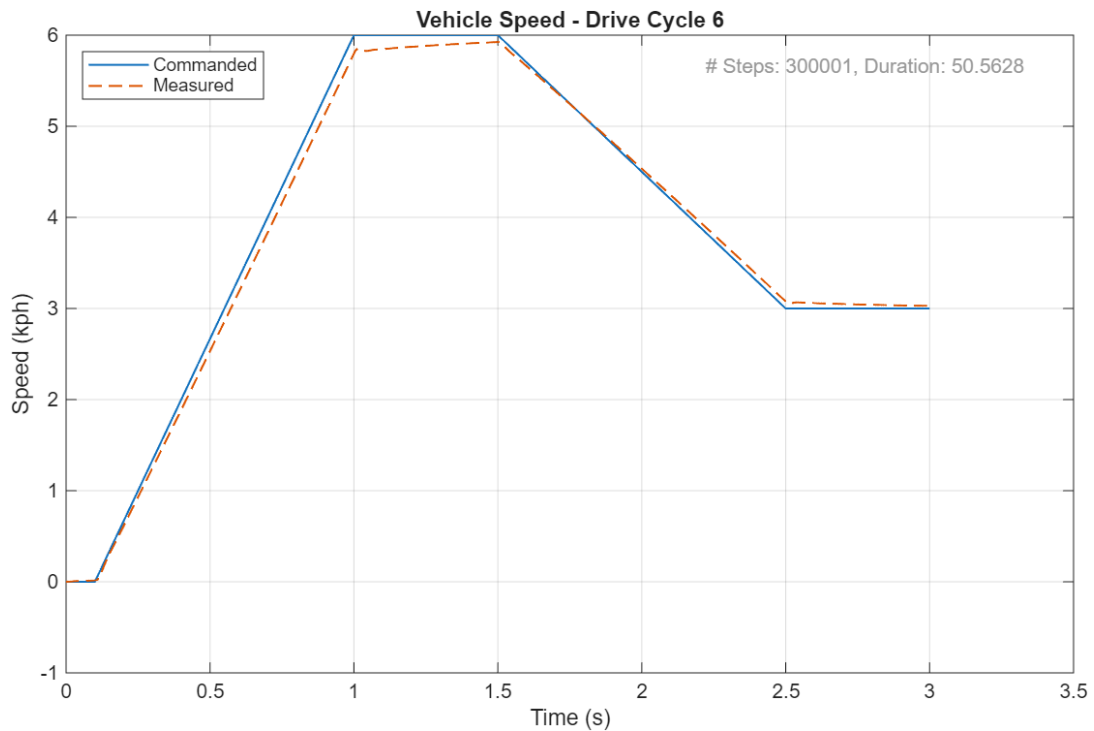


Figure 7.7: Drive Cycle 6, electrical system in Detailed variant. Compared to Figure 7.6, the measured curve no longer leads the commanded one and tracks the reference with a small physical lag during the acceleration ramp; the drop towards 3 kph after $t \approx 1.5$ s is followed cleanly. The improved tracking is the visible benefit of the higher-fidelity Detailed electrical variant on this short cycle.

The solver-step count printed in each figure ("# Steps") makes the cost difference explicit: the System variant runs at variable step with averaged dynamics and completes in a fraction of a second, while the Detailed variant produces hundreds of thousands of fixed-step samples and is two orders of magnitude slower in wall-clock time. For long drive cycles (1–5) this overhead is significant, which is why only cycle 6 is documented in Detailed; the trade-off is acceptable when localised electrical behaviour is the object of study.

8. Requirement Linking with the Requirements Manager

To close the loop between the requirement baseline and the executable model, the 23 low-level software requirements (SwLLR01 ... SwLLR23) were created in a Simulink Requirements set (HEV_powersplit_adapted_swllr.sreqx) and linked to the corresponding Stateflow transitions of HEV_powersplit_adapted/Control/Mode Logic. The work was scripted in Scripts_Data/link_swllr_requirements.m, which programmatically creates each requirement (with its CustomID, Summary, and Description) and then calls `sreq.createLink` against the matching transition handle. Junction-mediated transitions in the chart (the StandStill→Start path, the three Motion_mode_ICE→StandStill paths, and the Start→ICE path) are linked on both halves of the junction so that the requirement is associated with the full source-to-destination decision.

Once the script has run, the linked requirements are visible from the Simulink toolstrip, under Apps → Requirements Manager. The Requirements Manager view displays the chart on the canvas, the Requirements Editor with the SwLLR list at the bottom, and the Property Inspector on the right side showing the currently selected requirement, including its CustomID, Summary, Description (the "shall" statement extracted from Requirements.docx), and the link back to the model element.

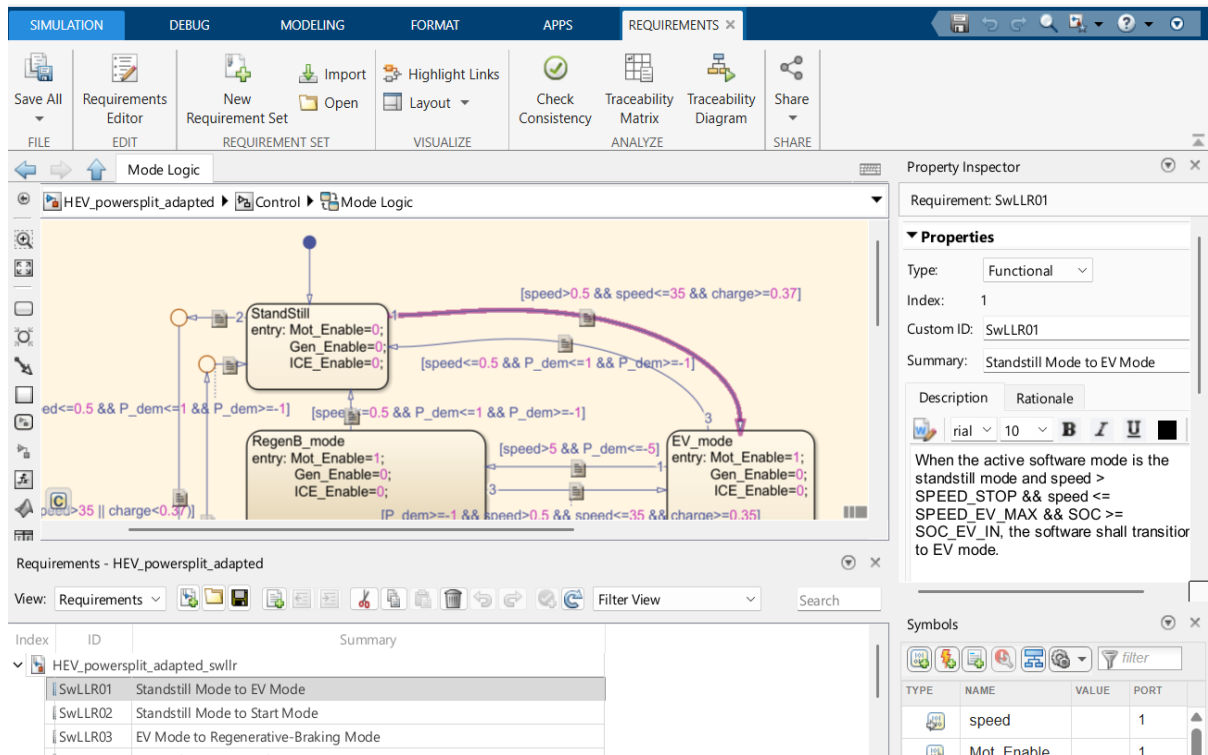


Figure 8.1: Requirements Manager view of the adapted model. The Mode Logic chart is shown on the canvas with the linked transitions highlighted; the Requirements Editor at the bottom lists the SwLLR requirements; the Property Inspector on the right shows the details of the currently selected requirement (Custom ID, Summary, Description).

From this view the user can navigate in either direction: clicking a transition in the chart opens its associated requirement in the Property Inspector, and clicking a requirement in the editor highlights the linked transition in the chart. This traceability is what allows the requirement baseline, the Stateflow implementation, and the future C implementation to be reviewed together as one consistent artefact.